Explore Functional Programming in Go

เวิร์กชอป 2 ชั่วโมง · กรอบ Actions / Calculations / Data

แนวคิดจาก Grokking Simplicity (Eric Normand) — แบบ pragmatic สำหรับ Go developer

อาทิตย์ที่ 18 ตุลาคม 2026 · โค้ดตัวอย่าง: ~/src/go-fp

แผนวันนี้ (2 ชม.)

มองโค้ด → เขียนโค้ด → ใช้จริง → ลงมือทำ

เวลา	เรื่อง	วิธี
0:00–0:15	เปิด: โค้ดปนเปื้อน 1 หน้า	แยก Action/Calculation/Data ร่วมกันบนกระดาน
0:15–0:40	Refactor live	ดึง calculation ออกจาก action → test ง่ายขึ้น
0:40–1:05	เครื่องมือของ Go	HOF, closures (+ pitfall), generics Map/Filter/Reduce
1:05–1:25	FP ในโค้ดจริง	Functional Options + middleware (= composition)
1:25–1:45	เวิร์กชอป	แยก coupon ออกจาก sending email ด้วยมือตัวเอง
1:45–2:00	ของหวาน + สรุป	iter.Seq lazy pipeline + cheat sheet

แกนคิดของวันนี้: Actions / Calculations / Data

ประเภท	คืออะไร	ตัวอย่าง	ชื่อ FP อื่นเรียก
Data	ข้อเท็จจริงของเหตุการณ์	email ที่ user กรอก, ยอดเงินจาก API	—
Calculation	input เดิม → output เดิม เสมอ	หาส่วนลด, validate email	pure function
Action	ผลขึ้นกับ 'รันกี่ครั้ง / เมื่อไหร่'	ส่ง email, เขียน DB, อ่านเวลา	side effect

- ทักษะที่ต้องการคือ "การมองโค้ด" (spotting) ก่อนเทคนิค — ใช้กับ l egacy Go ได้ทันที
- Go เป็นภาษาที่ act i on หลุดมาง่าย: mutation, goroutine, I/O อยู่ใกล้มือเสมอ
- **วันนี้ไม่มี monad / functor / type theory — มีแต่คำถามเดียว: โค้ดบรรทัดนี้ "พึ่งโลกภายนอก" ไหม?**

เปิด: โค้ดนี้ทำอะไรบ้าง?

จาก go-fp/01-acd-opener — ลองตีเส้นแยก: บรรทัดไหน Data / Calculation / Action?

0:00–0:15

```
func ProcessOrder(order *Order) {
    order.ProcessedAt = time.Now()          // ????
    fmt.Printf("processing %s ...", order.Email)
    order.Total = 0                          // ????
    for _, it := range order.Items {
        order.Total += it.Price * it.Quantity
    }
    if order.Total >= 1000 {
        order.Total = order.Total * 90 / 100 // ????
    }
    orders[order.Email] = *order             // ????
    fmt.Printf("saved: total=%d", order.Total)
}
```

กติกา

- อ่านเงียบ ๆ 1 นาที
- จับคู่ชี้บรรทัด 3 นาที
- เฉลยบนกระดานร่วมกัน

คำใบ้: บรรทัดเดียวอาจมีมากกว่า 1 ประเภทซ้อนกัน

Refactor live: ดึง Calculation ออกจาก Action

0:15–0:40

❌ ก่อน

```
// ❌ Action ปนCalculation
func ApplyDiscount(cart Cart) Cart {
    fmt.Println("applying discount")   // side effect
    cart.Total = cart.Total * 90 / 100 // mutate input
    return cart
}
```

✅ หลัง

```
// ✅ Calculation — pure, ทดสอบง่าย
func Discounted(total int) int {
    return total * 90 / 100
}

// action บาง ๆ ทำหน้าที่แค่ "เกิดขึ้น"
func LogDiscount(total int) {
    fmt.Println("discount", Discounted(total))
}
```

table-driven test — ไม่ต้อง mock, ไม่ต้อง capture stdout

```
func TestDiscounted(t *testing.T) {
    tests := []struct{ in, want int }{
        {1000, 900}, {999, 999}, {0, 0},
    }
    for _, tt := range tests {
        if got := Discounted(tt.in); got != tt.want {
            t.Errorf("Discounted(%d) = %d; want %d",
                tt.in, got, tt.want)
        }
    }
}
```

เครื่องมือของ Go (1): Higher-order + Closures

ฟังก์ชันเป็นค่าได้ — std lib ใช้เยอะแล้ว: sort.Slice, http.HandleFunc, context.WithCancel

0:40–1:05

```
// currying ด้วย closure
func Adder(a int) func(int) int {
    return func(b int) int { return a + b }
}

add10 := Adder(10)          // เก็บ "ครึ่งการคำนวณ" ไว้ก่อน
add10(5)                    // == 15

// generic curry 2 พารามิเตอร์
func Curry2[A, B, R any](f func(A, B) R) func(A) func(B) R {
    return func(a A) func(B) R {
        return func(b B) R { return f(a, b) }
    }
}
```

- closure = ฟังก์ชันที่จำบริบทตอนถูกสร้าง → นี่คือวิธี Go ทำ partial application
- แต่ระวัง: closure จับ 'ตัวแปร' ไม่ใช่ 'ค่า' → เจอเรื่องในสไลด์ถัดไป

Pitfall ที่ต้องรู้: loop variable (ก่อน Go 1.22)

0:40–1:05

```
for i := 0; i < 3; i++ {
    go func() { fmt.Println(i) }()
}
// Go 1.22+: พิมพ์ 0..2 ไม่ซ้ำ (loop var แยกต่อรอบ)
// Go 1.21-:  ส่วนใหญ่พิมพ์ 3 3 3 (ตัวแปรตัวเดียวถูกจับร่วม)
//
// วิธีเดิมก่อน 1.22:
for i := 0; i < 3; i++ {
    i := i // สำเนาต่อรอบ
    go func() { fmt.Println(i) }()
}
```

- จุดสอน: closure จับตัวแปรเดียวกัน ไม่ใช่สำเนา — ยกเว้น Go 1.22+ ที่ loop var แยกต่อรอบแล้ว
- โค้ดเก่าในทีม (ก่อน 1.22) ยังมีรอยแก้ i := i ทิ้งไว้ให้เห็นได้

เครื่องมือของ Go (2): Generics — Map / Filter / Reduce

0:40–1:05

Go 1.18+ · โค้ดเต็ม + TDD ใน go-fp/03-fp-toolkit

```
func Map[T, R any](in []T, f func(T) R) []R {
    out := make([]R, 0, len(in))
    for _, v := range in { out = append(out, f(v)) }
    return out
}

func Filter[T any](in []T, keep func(T) bool) []T { ... }

// accumulator ต่าง type กับสมาชิกได้ — จุดที่คนถามแน่
func Reduce[T, R any](in []T, init R,
    combine func(R, T) R) R { ... }

total := Reduce(items, 0, func(acc int, it Item) int {
    return acc + it.Price*it.Quantity
})
```

- Go ไม่มี built-in map/filter — เขียนเองได้ใน 5 บรรทัด แต่ Go community นิยม for loop ตรง ๆ: โค้ดชุดนี้คือเครื่องมือ 'เรียนรู้' ไม่ใช่แนวปฏิบัติ
- Reduce ที่ accumulator ต่าง type กับ element คือจุดที่ generics ช่วยจริง (Reduce[Item, int])

FP ในโค้ดจริง: Functional Options

closure + higher-order function ที่คุณใช้อยู่แล้วโดยไม่รู้ตัว

1:05–1:25

```
type Option func(*Server)

func WithPort(p int) Option {
    return func(s *Server) { s.port = p }
}

func New(host string, opts ...Option) *Server {
    s := &Server{host: host, port: 8080} // default ก่อน
    for _, o := range opts { o(s) }      // แล้ว override
    return s
}

srv := New("localhost", WithPort(9090))
// grpc-go, zap, aws-sdk-go-v2 ล้วนใช้ pattern นี้
```

- อ่านได้ว่า: WithPort(9090) คือ 'calculation ที่ผลิต action เล็ก ๆ' เอาไว้ยิงใส่ constructor ตอนหลัง
- **เห็นใน production จริง: grpc-go, uber-go/zap, aws-sdk-go-v2**

Middleware = Function Composition

1:05–1:25

⚠️ สไลด์ 'ตัดได้' ตัวแรก — เวลาไม่พอให้ข้ามไปเวิร์กชอปได้เลย

```
type Middleware func(http.Handler) http.Handler

func Logger(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s (%s)", r.Method, r.URL, time.Since(start))
    })
}

func Chain(mw ...Middleware) Middleware {
    return func(final http.Handler) http.Handler {
        for i := len(mw) - 1; i >= 0; i-- {
            final = mw[i](final)
        }
        return final
    }
```

- http.Handler เป็น function เดียว → wrapper ครอบซ้อน ๆ = composition แบบ Go
- อ่าน Chain(Logger, Recover) จากในออกนอก: ลำดับ = ความหมายทางธุรกิจ

เวิร์กชอป: Coupon Email

โจทย์จริงจาก Grokking Simplicity บท 3–5 (พอร์ตเป็น Go แล้ว: go-fp/07-coupon-email-exercise)

1:25–1:45

starter/coupons.go

```
// จุดเริ่ม: ทุกอย่างอยู่ในฟังก์ชันเดียว
func EmailCoupons(cart Cart, coupons []Coupon,
    out io.Writer) error {
    fmt.Printf("finding coupons for %s\n", cart.Email)
    var codes []string
    for _, c := range coupons {
        if cart.Total >= c.MinimumTotal &&
            (c.Category == "" || c.Category == cart.Category)
            codes = append(codes, c.Code)
        }
    }
    _, err := fmt.Fprintf(out,
        "To: %s\nCoupons: %v\n", cart.Email, codes)
    return err
}
```

โจทย์

1. จับคู่ — แยก A/C/D ให้ครบ (3 น.)
2. ดึง FindApplicableCoupons ออกมา + เขียน table-driven test ก่อน implementation (10 น.)
3. เหลือ SendCouponEmail เป็น action บาง ๆ (5 น.)
4. เทียบกับ solution/ (2 น.)

กติกา: เงินเป็นบาทเต็มไม่ติดลบ · coupon ไม่มีหมวด = ใช้ได้ทุกหมวด · รักษาลำดับเดิม

เฉลย + ของหวานปิดท้าย: Lazy Pipeline (Go 1.23+)

1:45–2:00

solution/

```
// Calculation — pure ทดสอบได้โดยไม่ต้อง mock อะไร
func FindApplicableCoupons(cart Cart,
    coupons []Coupon) []Coupon {
    result := make([]Coupon, 0)
    for _, c := range coupons {
        if cart.Total >= c.MinimumTotal &&
            (c.Category == "" || c.Category == cart.Category) {
            result = append(result, c)
        }
    }
    return result
}

// Action — บาง ๆ ทำหน้าที่เดียว
func SendCouponEmail(out io.Writer, email string,
    coupons []Coupon) error { ... }
```

09-lazy-iter-seq

```
// infinite sequence — ไม่มีเงื่อนไขหยุดในตัว
func Fibonacci() iter.Seq[int] {
    return func(yield func(int) bool) {
        a, b := 0, 1
        for {
            if !yield(a) { return } // ผู้บริโภคเป็นคนหยุด
            a, b = b, a+b
        }
    }
}


isEven := func(n int) bool { return n%2 == 0 }
even := Filter(Fibonacci(), isEven)
total := 0
for n := range Take(10, even) { // lazy คำนวณเท่าที่จำเป็น
    total += n
} // sum แรก 10 ตัวเลขคู่ = 257114
```

- calculation ที่ pure เต็มตัว = test ไม่ต้อง mock / action ที่บาง = เหลือแค่ 'จุดที่โลกเปลี่ยน'
- iter.Seq = lazy sequence ใน std lib: Fibonacci เป็นอนุกรมไม่รู้จบ แต่ Take(10) คำนวณแค่ที่จำเป็น

Cheat Sheet: ถาม–ตอบ 3 คำถามกับโค้ดทุกบรรทัด

ถาม	ถ้าคำตอบคือ...	แล้วทำอะไร
"ผลลัพธ์ขึ้นกับเวลา/จำนวนครั้งไหม?"	ใช่ → Action	ดันไปไว้ขอบระบบ ทำให้บางที่สุด ไม่ test ก็ได้
"input เดิม ให้ผลเดิมเสมอไหม?"	ใช่ → Calculation	ทำให้ pure + เขียน table-driven test
"เป็นแค่ข้อเท็จจริงที่ส่งต่อกันไหม?"	ใช่ → Data	ใช้ immutable เท่าทำได้ (value semantics)

- Go ทำได้ดี: HOF, closures, generics (1.18+), iter.Seq (1.23+)
- Go ไม่มี: built-in map/filter, immutability guarantee, tail-call optimization — รู้ขีดจำกัด แล้วออกแบบรอบมัน
- **คำโปรย: 'คุณไม่ต้องเขียน FP ทั้งวัน — แค่หมั่นย้าย calculation ออกจาก action ทีละฟังก์ชัน'**

อ่านต่อ

- Grokking Simplicity — Eric Normand (Manning 2021) · โฟกัส ch.3–5 (ACD), ch.6–7 (immutability), ch.10–14 (first-class functions)
- Functional Programming in Go — Dylan Meeus (Packt 2023)
- Go blog: Range Over Function Types (Go 1.23) — go.dev/blog/range-functions
- Codewalk: First-Class Functions in Go — go.dev/doc/codewalk/functions
- Eli Bendersky: Higher-order functions in Go — eli.thegreenplace.net
- **โค้ดทั้งหมดวันนี้: github.com/.../go-fp (หรือ ~/src/go-fp) — go test ./... ผ่านทุก package**

🙏 — □□□□□□□□□□□□□